

MODEL CONTEXT PROTOCOL (MCP)

The easiest way to remember MCP:

AI needs outside data or actions

|
MCP
|

AI connects to external systems in a standard way

Quick memory line: MCP is like USB-C for AI integrations.

Contents

1. What is MCP?
2. Why MCP matters
3. MCP architecture: Host, Client, Server
4. MCP layers and transports
5. MCP Server: Tools, Resources, Prompts
6. MCP Client: Elicitation and deprecated features
7. How MCP works end to end
8. Complete travel example
9. Quick revision sheet
10. Current-spec notes and official sources

1. What is MCP?

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems. It gives AI applications a common way to access data, use tools, and work with reusable workflows or prompts.

AI can connect to	Examples
Data sources	Files, databases, knowledge bases
Tools	Search engines, calculators, APIs, actions
Applications	Calendar, GitHub, Slack, Notion
Workflows	Reusable prompts and structured interactions

Without MCP:

User -> AI application

With MCP:

User -> AI application -> MCP -> Files / Database / Calendar / Tools

USB-C analogy: USB-C gives devices a standard connection. MCP gives AI applications a standard connection to external systems.

Important scope: MCP standardizes context exchange and tool/data connectivity. It does not tell an AI application which LLM to use or how it must manage the context internally.

2. Why MCP matters

Before MCP, developers often built separate custom integrations for every system. MCP provides a common protocol, reducing integration complexity and making integrations easier to reuse across compatible hosts.

Who benefits?	Benefit
Developers	Less custom integration code and faster integration work.
AI applications / agents	Access to more data sources, tools, and applications.
End users	AI can answer with external context and perform useful

Who benefits?	Benefit
	actions when allowed.

Example:

User: "What meetings do I have tomorrow?"

AI application -> MCP connection -> Calendar system -> Meeting data -> AI response

3. MCP architecture: Host, Client, Server

```

USER
|
MCP HOST (AI application)
|
MCP CLIENT
|
MCP SERVER
|
External data / tool / application

```

Component	Simple meaning	Example
MCP Host	The main AI application that the user interacts with. It manages one or more MCP clients.	Claude Desktop, Claude Code, VS Code
MCP Client	The protocol component that maintains a dedicated connection to one MCP server for the host.	A client connection created inside the host
MCP Server	A program that exposes context or capabilities through MCP.	Filesystem, database, GitHub, weather server

Precise connection rule: Inside a host, each connected MCP server gets its own MCP client connection. A remote MCP server can still serve many clients from many hosts.

One host using multiple servers:

```

AI Application / Host
+-- MCP Client 1 -> Filesystem Server
+-- MCP Client 2 -> Database Server
+-- MCP Client 3 -> Remote Sentry Server

```

Local vs remote servers: An MCP server can run on the same machine as the host or on a remote platform. "Server" describes its protocol role, not its physical location.

4. MCP layers and transports

Layer	Simple meaning	Main responsibility
Data layer	What the client and server say to each other.	JSON-RPC messages, discovery, tools, resources, prompts, notifications
Transport layer	How those messages travel.	Connection, framing, authentication,

Layer	Simple meaning	Main responsibility
		secure communication

Data layer: MCP uses JSON-RPC 2.0 as its underlying request/response message format.

Transport layer: The 2026-07-28 documentation describes two main transports:

- **STDIO** - mainly for local process-to-process communication on the same machine.
- **Streamable HTTP** - mainly for remote communication; supports HTTP authentication and optional streaming.

Current transport note: Legacy HTTP+SSE is deprecated. New remote implementations should use Streamable HTTP.

Statelessness: In MCP 2026-07-28, every request carries protocol/version and relevant capability metadata so a server can process that request independently.

5. MCP Server: Tools, Resources, Prompts

These are the three core server primitives. This is one of the most important MCP concepts.

Primitive	Remember it as	What it does	Who controls it
Tools	DO	Executable functions that perform actions.	Model
Resources	READ / KNOW	Read-only contextual data the application can retrieve.	Application
Prompts	GUIDE	Reusable instruction templates.	User

5.1 Tools = actions

A tool is an executable function that the AI model can choose to call based on the user request.

Examples:

```
searchFlights()
sendEmail()
createCalendarEvent()
queryDatabase()
```

Method	Purpose
tools/list	Discover available tools and their schemas.
tools/call	Execute a specific tool with arguments.

Human control: Tools are model-controlled, but the application can require approvals, permissions, or activity logging before sensitive actions are executed.

5.2 Resources = contextual data

A resource is information that the AI application can retrieve and provide to the model as context. Resources are passive/read-only from the MCP primitive perspective.

Examples:

File contents
Database schema or records
Calendar data
API documentation
Knowledge-base documents

Resource type	Meaning	Example
Direct Resource	Fixed URI pointing to specific data.	calendar://events/2026
Resource Template	Dynamic URI with parameters.	weather://forecast/{city}/{date}

Method	Purpose
resources/list	List direct resources.
resources/templates/list	List resource templates.
resources/read	Read resource contents.
subscriptions/listen	Optionally listen for resource updates.

5.3 Prompts = reusable instructions

Prompts are parameterized, reusable instruction templates exposed by an MCP server. They are user-controlled and are explicitly selected/invoked rather than automatically triggered by the model.

Example prompt: plan-vacation

Inputs: destination, duration, budget, interests

Method	Purpose
prompts/list	Discover available prompts.
prompts/get	Retrieve a prompt and its arguments/details.

Best memory trick: Tool = DO. Resource = KNOW. Prompt = GUIDE.

6. MCP Client: Elicitation and deprecated features

The MCP client mainly exists to communicate with its MCP server. Clients can also expose features that allow richer server-to-user workflows.

6.1 Elicitation = ask the user for missing information

Elicitation allows an MCP server to request information from the user through the client while a task is in progress.

Server needs more information

|

```

MCP Client asks User
|
User provides input
|
Client retries / continues the server request

```

Mode	Use it for	Example
Form mode	Normal structured information collected in the client UI.	Seat preference, room type, confirmation
URL mode	Sensitive or external flows that should happen outside the client.	OAuth authorization, credential entry

Security rule: Do not request passwords, API keys, access tokens, or payment credentials through elicitation form mode. Use URL mode for sensitive flows so the secret does not pass through the client or LLM context.

6.2 Deprecated client features in 2026-07-28

Feature	What it meant	Current guidance
Roots	Client told a server which filesystem directories to focus on.	Deprecated. Pass files/directories using tool parameters, resource URIs, or server configuration.
Sampling	Server asked the client to perform an LLM completion on its behalf.	Deprecated. New implementations should integrate directly with an LLM provider API when model inference is needed.
Logging	Protocol-level server-to-client logging feature.	Deprecated. Use stderr for stdio or OpenTelemetry instead.

Deprecated does not mean removed: Roots, Sampling, and Logging still work during the deprecation window, but new implementations should not adopt them. The 2026-07-28 release states a minimum twelve-month deprecation window.

Roots security note: Roots were guidance for scope, not a security boundary. Real file security must come from operating-system permissions, sandboxing, or similar controls.

7. How MCP works end to end

A typical MCP interaction looks like this:

1. The host creates an MCP client for a configured MCP server.
2. The client may call server/discover to learn the server identity, supported protocol versions, and capabilities.
3. The client discovers available primitives, for example tools/list, resources/list, or prompts/list.
4. The AI application makes the relevant tools/context available to the model.
5. The user sends a request.
6. The model decides whether a tool is needed.
7. If needed, the MCP client sends tools/call to the correct MCP server.
8. The server performs the operation and returns structured content.

9. The result is given back to the model as context.
10. The model produces the final answer or decides on another step.

Discovery nuance: Every server must implement server/discover, but a client is not required to call it before every operation. A client may send another request directly and handle a protocol-version error if necessary.

7.1 Notifications = server tells the client something changed

MCP supports opt-in real-time notifications. A client can open a subscriptions/listen stream for changes it cares about.

```
Example:
Tool list changes on the server
|
Server sends notifications/tools/list_changed
|
Client calls tools/list again
|
AI application refreshes available tools
```

Important: Notifications are best-effort. Clients may still need polling or refresh logic to preserve freshness after reconnects.

8. Complete travel example

This single example combines Host, Clients, Servers, Resources, Prompts, Tools, and Elicitation without repeating the concepts.

```
USER
| "Plan a 7-day Barcelona vacation"
v
TRAVEL AI HOST
|
+-- MCP Client -> Travel Server
+-- MCP Client -> Weather Server
+-- MCP Client -> Calendar / Email Server
```

1. The user selects a reusable Plan Vacation prompt and provides destination, dates, budget, and interests.
2. The application retrieves useful Resources such as calendar availability and previous travel preferences.
3. The model chooses Tools such as searchFlights() and checkWeather().
4. Before a sensitive action such as booking, the server can use Elicitation to ask for confirmation or missing preferences.
5. After approval, tools such as bookHotel(), createCalendarEvent(), or sendEmail() can be executed.
6. The final result is returned to the user with the gathered context and completed actions.

```
PROMPT    -> What workflow should guide the AI?
RESOURCE  -> What information does the AI need?
TOOL      -> What action should the AI perform?
ELICITATION -> What missing input/approval is needed from the user?
```

9. Quick revision sheet

Term	One-line meaning
MCP	Standard protocol for connecting AI applications to external systems.
Host	Main AI application the user interacts with.
Client	Protocol component connecting the host to one MCP server.
Server	Program that exposes MCP capabilities.
Tool	Executable action - DO.
Resource	Read-only contextual data - KNOW.
Prompt	Reusable instruction template - GUIDE.
Elicitation	Server asks user for missing information through the client.
Data layer	Defines message semantics using JSON-RPC 2.0.
Transport layer	Defines how MCP messages travel.
STDIO	Common local transport.
Streamable HTTP	Current remote transport.

30-second recap: MCP Host manages MCP Clients. Each client talks to one MCP Server. Servers expose Tools, Resources, and Prompts. Tools perform actions, Resources provide context, and Prompts provide reusable instructions. Elicitation lets a server ask the user for missing input. MCP uses JSON-RPC at the data layer and STDIO or Streamable HTTP for transport.

10. Current-spec notes and official sources

This guide was checked against the official MCP documentation for protocol version 2026-07-28. The most important current-version notes are:

- Roots, Sampling, and protocol Logging are deprecated, not immediately removed.
- Legacy HTTP+SSE is deprecated; Streamable HTTP is the current remote transport.
- Elicitation remains the main active client primitive described in the client concepts page.
- MCP 2026-07-28 uses stateless per-request metadata and server/discover for version/capability discovery.

Official sources:

- [What is MCP?](#)
- [Architecture overview](#)
- [Understanding MCP servers](#)
- [Understanding MCP clients](#)
- [2026-07-28 changelog](#)

Prepared as a beginner-friendly teaching/study guide. Wording is intentionally simplified, while technical claims are aligned with the cited official MCP documentation.